

WacallerAPI — Enterprise Production Architecture & Implementation Plan

Document purpose: This document is the authoritative implementation plan for transforming the existing WacallerAPI repository from its current working prototype into a secure, robust, multi-tenant, production-ready communications platform.

Implementation agent: Antigravity

Execution model: Phase-by-phase implementation with verification gates after every phase.

Maximum phases: 10.

0. Mission and Non-Negotiable Requirements

You are working on an existing, functioning WacallerAPI system.

Do not treat this as a greenfield rewrite.

The existing repository contains a technically substantial WhatsApp VoIP engine, including:

- WhatsApp connectivity through [whatsmeow](#)
- WhatsApp multi-device session handling
- custom signaling
- call state management
- RTP
- SRTP
- STUN
- WebRTC/Pion
- relay transport
- MLow codec implementation
- PCM audio handling
- WebSocket audio streaming
- WebRTC browser bridging
- text/media/location messaging
- webhook infrastructure
- REST API
- API-key authentication
- billing/Stripe integration
- SQLite persistence

- embedded frontend/dashboard
- OpenAPI/Swagger support

The existing VoIP/media subsystem is the technically most sophisticated part of the system. The goal is **not to replace it unnecessarily**.

The primary objective is to build a proper **enterprise-grade platform around the existing communications engine**.

The final architecture must provide:

Secure Authentication

+

Strong Multi-Tenant Authorization

+

Reliable Billing/Entitlements

+

Durable Events/Webhooks

+

WhatsApp Session Ownership

+

Real-Time Worker Architecture

+

Production Infrastructure

+

Observability

+

Disaster Recovery

+

Security Testing

while preserving the existing WhatsApp/VoIP functionality.

1. Current System — What Exists Today

The current repository contains approximately:

- 101 Go source files
- approximately 24,300 lines of Go code
- embedded frontend
- approximately 33 Go test files

The existing architecture and findings were reviewed directly from the repository. The current production-readiness document identifies the major platform weaknesses, including billing bypass, missing object-level authorization, unauthenticated session lifecycle operations,

weak password storage, permanent PATs, XSS risk, SSRF, missing rate limiting, and hardcoded superadmin credentials.

1.1 Existing VoIP Engine

Current subsystem:

internal/voip/

core/

call/

media/

signaling/

transport/

wanode/

It contains:

- MLow codec
- RTP
- SRTP
- STUN
- WebRTC/Pion
- signaling
- relay transport
- call state machine
- media processing
- PCM handling

This subsystem is the core technical asset of the project.

Required treatment

Preserve it.

Do not rewrite the VoIP engine merely to make the architecture look cleaner.

However, it is **not exempt from production-readiness testing**.

The following must be validated:

- SRTP authentication
- SRTP replay protection
- ROC handling
- RTP sequence handling
- malformed packet handling
- packet reordering
- duplicate packets

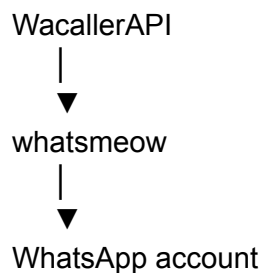
- sequence rollover
- race conditions
- memory/resource exhaustion
- codec regression
- signaling regression
- relay failure/recovery

Any modification to protocol or codec behavior requires regression tests.

2. Existing WhatsApp Integration

The current system uses **whatsmeow** and its SQL store for WhatsApp device/session credentials.

The current design is effectively:



This is retained.

However, the WhatsApp credential store must be treated as highly sensitive because losing or compromising it directly affects customers' linked WhatsApp accounts.

The production architecture must therefore provide:

- isolated storage
- restricted database access
- encryption at rest where appropriate
- secure backups
- restore testing
- no credential values in logs
- no credential values in API responses
- no credential values in the admin dashboard
- auditing of privileged access
- secure deletion
- controlled worker access

The existing document already recommends separating the WhatsApp device store from application data.

3. Existing API

The current backend uses:

Go
net/http
ServeMux

with public/protected routers.

The API currently exposes functionality including:

- authentication
- user registration/login
- session creation
- QR retrieval
- pairing
- session logout/deletion
- text messaging
- media messaging
- location messaging
- call creation
- call control
- call history
- audio playback
- WebSocket audio
- WebRTC audio
- API keys
- billing
- admin operations
- system statistics
- webhooks
- OpenAPI/Swagger
- compatibility endpoints

The current API layer is overly centralized, especially [internal/api/routes.go](#).

It must be decomposed into resource-oriented handlers and application services.

4. Existing Database

The current application uses SQLite.

The current SQLite database contains both:

Application data

- users
- API keys
- subscriptions
- sessions
- calls/messages/etc.

WhatsApp device data

- whatsmeow store

The current production-readiness document correctly identifies this as suitable for the prototype but unsuitable as the primary database architecture for a horizontally scalable multi-tenant SaaS.

The target architecture is:

PostgreSQL

- users
- organizations
- memberships
- API credentials
- WhatsApp session metadata
- calls
- messages
- subscriptions
- entitlements
- audit logs
- webhook events
- outbox
- usage

and separately:

WhatsApp device store

5. Existing Frontend

The current frontend is a single embedded HTML application inside the Go binary.

It currently provides:

- login
- registration

- dashboard
- session management
- messaging/calling functionality
- API-key management
- Swagger
- administrative functionality

The existing document identifies approximately 152 KB of embedded HTML and direct **innerHTML** usage as an important security and maintainability problem.

The target frontend architecture is:

Public Marketing Site

+

Customer Dashboard

+

Superadmin Operations Dashboard

These must have clearly separated authorization/security boundaries.

6. Existing Billing

Current system:

Stripe Checkout

+

direct/simulated activation path

The direct activation path is currently unsafe.

The existing analysis identified:

- direct client-controlled plan activation
- free billing upgrade path
- missing Stripe webhook verification
- insufficient billing integrity

as critical problems.

The final system must make Stripe-derived billing state authoritative.

7. Existing Infrastructure

Current deployment:

Single Go binary

+

single VPS

+

.env

+

SQLite

The current system does not have:

- proper CI/CD
- production secrets management
- containerized deployment
- structured metrics
- distributed tracing
- formal audit logging
- tested disaster recovery
- horizontal worker architecture
- zero-downtime deployment strategy

The current document identifies these limitations explicitly.

8. Critical Security Findings That Must Be Closed

These findings are considered **known vulnerabilities**, not theoretical risks.

8.1 Free billing bypass

The current direct plan activation endpoint allows users to grant themselves paid plans.

This must be removed from all customer-accessible paths.

8.2 Unverified Stripe webhooks

Stripe webhook events are currently trusted without proper signature verification.

The final system must:

Raw request body



Stripe signature verification



Event parsing



Event ID deduplication



Transactional processing

Never process an unverified Stripe event.

8.3 Unauthenticated WhatsApp session lifecycle

The current session create/QR/pair/logout/delete paths can be reached without proper authentication because of the optional-auth routing design.

The current analysis found that anonymous requests could potentially operate on sessions when the session ID is known.

Final rule:

Every WhatsApp session operation



authenticated principal



organization membership



resource authorization

No anonymous session management.

8.4 Broken Object-Level Authorization

Messaging and calling handlers currently authenticate the caller but do not consistently verify that the target session belongs to that caller.

This is the highest-priority authorization issue.

Every resource must be authorized.

Never do:

sessionID

↓
getSession()

without:

principal

↓
organization

↓
authorization

↓
session

8.5 API-key deletion authorization

API-key deletion must verify ownership or administrative privilege.

No resource may be deleted solely because the caller knows the resource ID.

8.6 Weak password hashing

Current password hashing uses unsalted single-round SHA-256.

Replace with:

Argon2id

or a properly configured password-hashing algorithm such as bcrypt.

Existing passwords must be migrated through a forced reset because the old hashes cannot safely be converted into strong hashes.

8.7 Permanent PAT used as browser authentication

The current PAT is:

- permanent
- unscoped
- used for browser authentication
- used for API access
- stored in frontend `localStorage`
- difficult to revoke independently

The final system must completely separate:

Browser Authentication Session

from:

Programmatic API Credential

8.8 XSS

The existing frontend uses direct `innerHTML` rendering of server-controlled values.

Replace with a framework that provides safe escaping by default.

Never render untrusted server data directly into HTML.

8.9 SSRF

The current system performs server-side requests against user-supplied URLs.

This affects:

- audio URLs
- media URLs
- webhook URLs

The final system must implement a centralized secure outbound HTTP subsystem.

Requirements:

- HTTPS by default
- DNS resolution validation
- block loopback
- block private IPv4
- block link-local
- block metadata endpoints
- block private/link-local IPv6
- validate every redirect
- maximum redirects
- maximum response size
- connection timeout
- TLS timeout
- total request deadline
- controlled headers
- no credential leakage in logs

Where possible, replace arbitrary URL ingestion with direct upload to object storage.

8.10 Rate limiting

The current system lacks meaningful rate limiting.

Implement:

IP-based rate limiting

+

account/organization rate limiting

+

API credential rate limiting

+

resource-specific limits

+

concurrency limits

+

usage quotas

This is particularly important because WacallerAPI can place real communications calls.

9. New Target Architecture

The final architecture must explicitly separate the system into two major planes.

9.1 Control Plane

Responsible for:

Authentication

Organizations

Memberships

RBAC

Users

API credentials

Billing

Entitlements

Session metadata

Call metadata

Messages metadata

Webhooks
Audit
Usage
Admin
Feature flags

The control plane does **not** directly own the low-level WhatsApp real-time connection.

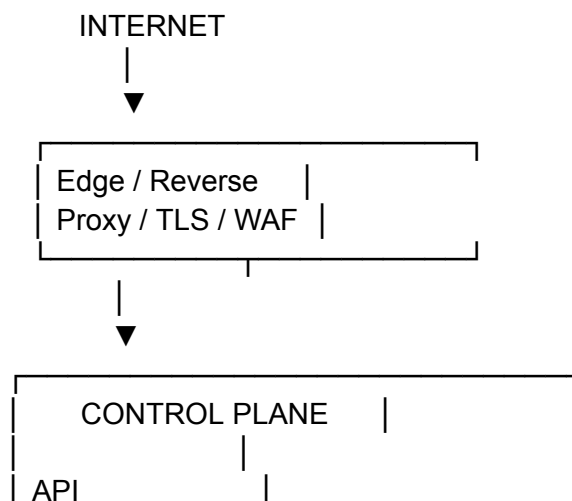
9.2 Real-Time Plane

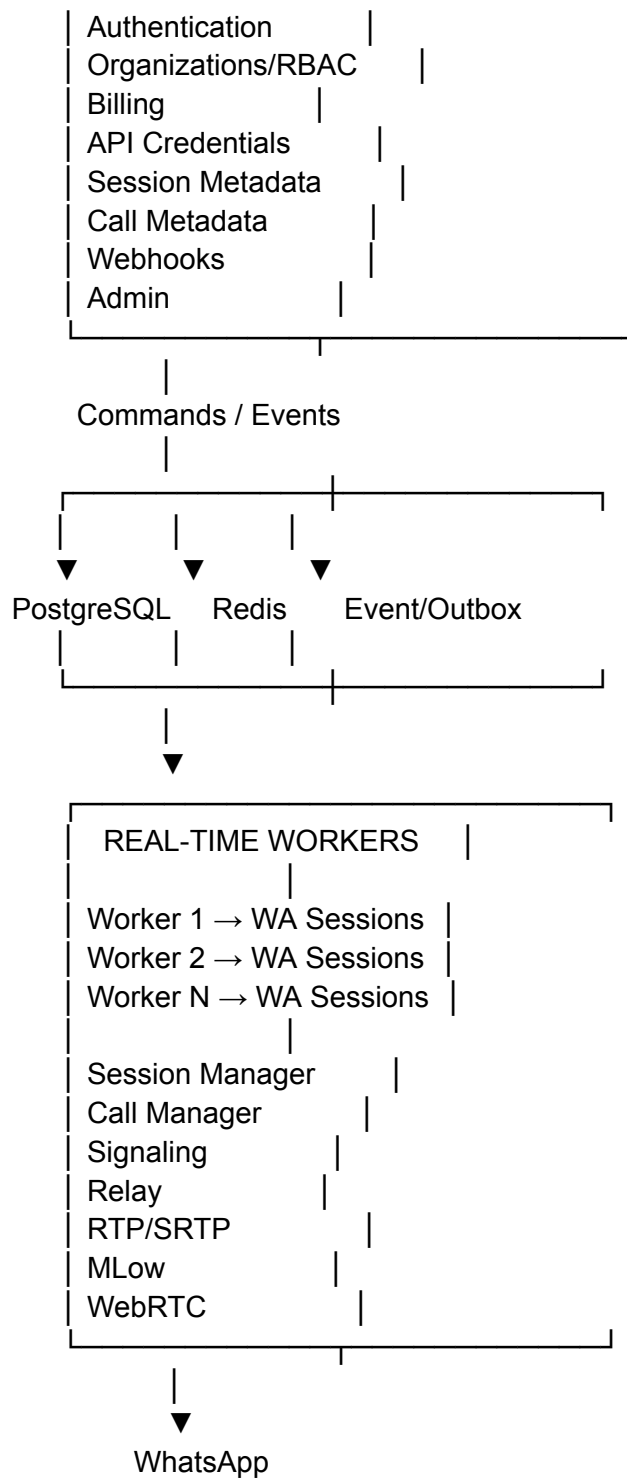
Responsible for:

WhatsApp connections
WhatsApp session lifecycle
Signaling
Call state
Relay
RTP
SRTP
STUN
MLow
PCM
WebRTC
WebSocket audio

The real-time plane is latency-sensitive and stateful.

9.3 Target Architecture Diagram





10. Multi-Tenant Architecture

Do not model the final system as merely:

User → resources

Introduce:

User



Organization



A user can belong to multiple organizations.

An organization is the primary ownership boundary for billable and operational resources.

11. Organization Roles

Initial RBAC model:

Organization Owner

Organization Admin

Developer

Viewer

Superadmin

Example permissions:

Resource	Owner	Admin	Developer	Viewer	Superadmin
Organization settings	✓	✓	—	—	✓
Members	✓	✓	—	—	✓
WhatsApp sessions	✓	✓	✓	Read	✓
Messaging	✓	✓	✓	—	✓
Calling	✓	✓	✓	—	✓

API credentials	✓	✓	✓	—	✓
Billing	✓	✓	—	—	✓
Audit logs	✓	✓	—	Read	✓
Platform administration	—	—	—	—	✓

Authorization must be **deny-by-default**.

12. Domain Model

The target application should conceptually contain:

User
 Organization
 Membership
 Identity
 AuthSession
 APIKey / APICredential
 WhatsAppSession
 WorkerLease
 Call
 Message
 WebhookEndpoint
 WebhookDelivery
 Subscription
 Entitlement
 UsageRecord
 AuditEvent
 OutboxEvent
 FeatureFlag

Do not create a generic "Session" abstraction that mixes:

- browser sessions
- API credentials
- WhatsApp sessions
- call sessions

These are different concepts.

13. Authentication Architecture

Browser authentication

Use a secure server-side session or carefully designed short-lived access/rotating refresh architecture.

If JWT is used, explicitly implement:

- short access lifetime
- rotating refresh tokens
- refresh-token reuse detection
- key rotation
- issuer validation
- audience validation
- signing algorithm restrictions
- expiration
- revocation
- secure storage

For a first-party dashboard, a server-side session cookie is also acceptable and may be simpler.

Browser credentials must never be stored in `localStorage`.

Use:

Secure
HttpOnly
SameSite

cookies.

Implement CSRF protection and origin validation for state-changing browser operations.

14. Email Authentication

Implement:

Registration
↓
Verification email
↓
Verified account

Also:

Forgot password



single-use token



short expiration



password reset

Add:

- login backoff
 - account protection against brute force
 - suspicious login monitoring
 - session listing
 - logout-current-device
 - logout-all-devices
-

15. Google Authentication

Implement Google OIDC using Authorization Code + PKCE.

Validate:

- state
- nonce
- issuer
- audience
- redirect URI
- authorization code
- token signature/claims

Support:

Password account

Google account

Password + Google account

with secure account linking.

16. MFA

Regular users:

TOTP MFA

available/optionally enforced depending on product policy.

Superadmins:

MFA mandatory

Sensitive admin operations require step-up authentication.

17. API Credentials

Personal/programmatic API credentials must be separate from browser sessions.

Each credential should contain:

id
organization_id
name
hash
prefix
scopes
created_at
expires_at
last_used_at
revoked_at

Never store the raw secret.

Never display the raw secret after initial issuance.

Example scopes:

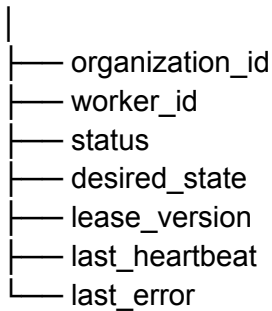
sessions:read
sessions:write
messages:read
messages:send
calls:read
calls:create
calls:control
webhooks:read
webhooks:manage
billing:read

18. Session Ownership

WhatsApp sessions are stateful.

Introduce a session registry:

WhatsAppSession



A worker acquires a lease.

Only the worker owning the lease may operate the live WhatsApp connection.

19. Worker Lease Architecture

Use a lease/fencing mechanism.

Session



Acquire lease



Worker A



Heartbeat

If Worker A dies:

heartbeat expires



lease expires



Worker B acquires new lease



Worker B reconnects WhatsApp

Prevent split-brain using lease versions/fencing.

Never allow two workers to believe they own the same WhatsApp session simultaneously.

20. Worker Lifecycle

Define explicit states:

CREATED
STARTING
CONNECTING
CONNECTED
DEGRADED
RECONNECTING
STOPPING
STOPPED
FAILED

Persist meaningful lifecycle metadata.

This state must be visible to the admin dashboard.

21. Event Architecture

Introduce domain events.

Examples:

UserRegistered
OrganizationCreated

SessionCreated
SessionConnecting
SessionConnected
SessionDisconnected
SessionPairingRequired
SessionLoggedOut

MessageQueued
MessageSent
MessageFailed
MessageReceived

CallCreated
CallRinging
CallConnected

CallEnded
CallFailed

SubscriptionChanged
WebhookDeliveryFailed

Each event should have:

event_id
organization_id
aggregate_type
aggregate_id
event_type
schema_version
occurred_at
metadata

22. Commands

Commands represent requested actions:

CreateSession
LogoutSession
DeleteSession

SendMessage
StartCall
AcceptCall
RejectCall
EndCall

PlayAudio

Commands should contain:

command_id
organization_id
target_id
requested_by
created_at
deadline

23. Transactional Outbox

All important state transitions that generate events must use the transactional outbox pattern.

Example:

```
BEGIN TRANSACTION
```

```
UPDATE calls  
SET status = 'ended'
```

```
INSERT INTO outbox_events (...)
```

```
COMMIT
```

Then:

Outbox Worker



Event consumers



Webhook / Audit / Metrics / Other services

This prevents database state and event state from diverging.

24. Webhook Architecture

Webhook delivery must be durable.

States:

```
PENDING  
DELIVERING  
DELIVERED  
RETRY_WAIT  
FAILED  
DEAD_LETTER
```

Persist:

```
delivery_id  
event_id  
endpoint_id  
attempt_count  
next_attempt_at  
last_status  
last_error
```

created_at
delivered_at

Use exponential backoff with jitter.

Never silently drop events because an in-memory queue is full.

25. Webhook Security

Webhook requests must include:

delivery ID
timestamp
signature

Sign a canonical representation such as:

timestamp + "." + delivery_id + "." + body

using HMAC.

Receivers must be able to reject:

- invalid signatures
 - stale timestamps
 - replayed deliveries
-

26. API Idempotency

All important state-changing endpoints should support:

Idempotency-Key: <unique-key>

Especially:

POST /sessions
POST /messages
POST /calls
POST /calls/{id}/actions

Store:

organization_id

idempotency_key
request_hash
status
response
created_at
expires_at

Ensure one idempotency key creates one logical operation.

27. Rate Limits, Quotas and Concurrency

Implement three separate controls.

Rate limits

Requests per time period.

Quotas

Total usage over billing periods.

Concurrency limits

Simultaneous expensive operations.

Example organization limits:

max WhatsApp sessions
max concurrent calls
max messages/minute
max API requests/minute
max WebSocket streams
max webhook endpoints
max media size

Do not rely on rate limiting alone.

28. Communications Abuse Prevention

Because WacallerAPI can place real communications calls, implement:

per-organization call rate

per-session call rate
concurrent call limits
abnormal usage detection
temporary suspension thresholds
admin alerts
feature kill switches

The system must be designed against:

- account compromise
 - API-key abuse
 - automated call flooding
 - messaging abuse
 - toll/fraud-like behavior
 - WhatsApp account abuse
-

29. Media Architecture

Prefer:

Client
↓
Object Storage
↓
WacallerAPI
↓
Validated Media
↓
WhatsApp

rather than requiring WacallerAPI to fetch arbitrary URLs.

Use S3-compatible storage.

For uploaded media:

- maximum size
 - MIME validation
 - magic-byte validation
 - safe filenames
 - malware/content scanning strategy
 - lifecycle policy
 - encryption
 - retention policy
-

30. Secure Outbound HTTP

All server-side HTTP requests must go through a centralized component.

Example:

OutboundHTTPClient

No direct:

http.Get(userURL)

in business logic.

It must provide:

SSRF protection
timeouts
redirect validation
response-size limits
DNS/IP validation
TLS configuration
controlled headers
logging/redaction

31. WebSocket Architecture

WebSocket audio must:

- authenticate before upgrade
- authorize organization/session/call
- validate origin
- enforce message size limits
- enforce connection limits
- use read/write deadlines
- implement ping/pong
- handle backpressure
- clean up on call termination
- avoid unbounded buffers
- prevent unauthorized cross-tenant subscriptions

Do not route raw audio through Redis.

Redis is for coordination/control-plane data, not high-volume media transport.

32. WebRTC Architecture

WebRTC signaling must:

- authenticate
 - authorize
 - validate SDP
 - limit SDP size
 - validate connection lifecycle
 - timeout stalled ICE connections
 - clean up dead connections
 - restrict DataChannel message types
 - enforce per-tenant connection limits
-

33. VoIP/Media Security Requirements

The existing VoIP engine must pass a dedicated security/reliability gate.

SRTP

Verify authentication tags before accepting packets.

Correct order:

```
Receive
↓
Parse
↓
Determine packet index
↓
Verify authentication
↓
Replay check
↓
Decrypt
↓
Update security state
↓
Decode
```

Do not mutate ROC/replay state based on unauthenticated packets.

Implement replay protection.

Test:

- forged packets
 - invalid authentication tags
 - duplicate packets
 - reordered packets
 - sequence rollover
 - ROC rollover
 - malformed packets
-

34. Database Architecture

Use PostgreSQL as the primary application database.

Use real versioned migrations:

001_initial.sql
002_organizations.sql
003_auth.sql
004_sessions.sql
...

Never use swallowed migration errors.

Migrations must fail startup when an essential migration fails.

35. Database Integrity

Use:

- foreign keys
- unique constraints
- check constraints
- indexes
- transactions
- appropriate cascading behavior
- optimistic/pessimistic concurrency where required

Important invariants must be enforced at the database level whenever practical.

36. Suggested Core Tables

At minimum:

users
organizations
memberships
identities
auth_sessions
api_credentials

whatsapp_sessions
worker_leases

calls
messages

subscriptions
entitlements
usage_records

webhook_endpoints
webhook_deliveries

outbox_events
audit_events

feature_flags

Exact schema can be refined during Phase 1/2 implementation.

37. Billing Architecture

Stripe must be the authoritative external billing system.

Architecture:

Stripe
↓
Verified webhook
↓
Billing Synchronizer
↓
Subscription

↓
Entitlements
↓
Usage enforcement

Never allow a customer to directly set:

plan = business

38. Subscription Model

Separate:

Subscription
Entitlement
Usage
Administrative Override

An administrative override must contain:

organization
feature
limit
reason
created_by
created_at
expires_at

Manual upgrades must never become uncontrolled permanent database mutations.

39. Stripe Requirements

Implement:

- official Stripe SDK
- webhook signature verification
- event-ID deduplication
- subscription synchronization
- cancellation
- payment failure handling
- dunning behavior
- proration
- reconciliation job

- subscription-state monitoring

No Stripe failure may result in automatic free entitlement.

40. Superadmin Architecture

Superadmin is a separate security tier.

Requirements:

mandatory MFA
short session lifetime
step-up authentication
strong audit logging
no normal API-key authentication
no public registration

First admin:

wacaller bootstrap-admin

or equivalent secure provisioning command.

No hardcoded admin account.

No predictable password.

No static token.

41. Superadmin Dashboard

The superadmin dashboard must support:

User/organization management

- search
- filtering
- organization details
- user details
- subscription
- usage
- sessions

- suspension
- deletion

Billing

- subscription state
- Stripe IDs/status
- entitlement state
- manual grants
- manual revocation
- billing anomalies

WhatsApp sessions

- connected sessions
- health
- worker
- last activity
- reconnect status
- force logout
- force delete

Never display WhatsApp credentials.

API credentials

Display:

- credential ID
- name
- owner
- scopes
- created
- last used
- expiration
- status

Never display raw credentials.

Webhooks

- endpoint
- status
- deliveries
- failures
- retry state
- dead letters

Audit

Search/filter:

actor
organization
action
target
timestamp
IP
result

Platform health

Show:

API status
worker status
DB status
Redis status
connected sessions
active calls
request rates
errors
latency
webhook queue
memory
goroutines

Feature flags

Allow controlled:

calling disable
messaging disable
tenant disable
platform-wide emergency disable

All sensitive operations must be audited.

42. Admin Impersonation

If support impersonation is implemented:

Support Access Session

- target organization
- target user
- reason
- created_by
- created_at
- expires_at
- scope

The interface must clearly indicate:

YOU ARE ACTING AS...

Prefer read-only impersonation.

Never expose or reuse the customer's API credential.

43. Audit Logging

Sensitive operations must create immutable audit records.

At minimum:

- actor
- organization
- action
- target
- result
- timestamp
- IP
- user-agent
- request_id
- reason
- metadata

Never store secrets inside audit records.

Never store:

- passwords
- raw PATs
- raw API keys
- cookies
- OAuth tokens
- WhatsApp credentials
- webhook secrets

44. Observability

Implement:

Logs

Structured JSON logs.

Metrics

At minimum:

API requests

API latency

API errors

active WhatsApp sessions

session reconnects

session failures

active calls

calls started

calls connected

calls failed

call duration

RTP packets

SRTP authentication failures

packet loss

codec failures

webhook deliveries

webhook failures

webhook retries

webhook queue depth

DB latency

DB errors

worker state

worker failover

Tracing

Use OpenTelemetry-style tracing for control-plane operations and important worker/call lifecycle operations.

Do not trace every RTP packet.

45. Correlation IDs

Every request must receive:

request_id

This should flow through:

HTTP
↓
Application Service
↓
Command
↓
Worker
↓
Call
↓
Webhook

A customer support investigation should be able to trace:

request_id
↓
command_id
↓
call_id
↓
session_id
↓
worker_id
↓
failure

46. Health Checks

Implement:

/live

/ready

/live

Checks process liveness.

/ready

Checks whether the service can actually accept work.

Readiness should account for required infrastructure such as:

PostgreSQL

Redis

required application initialization

47. Security Headers

Implement appropriate:

Content-Security-Policy

X-Content-Type-Options

X-Frame-Options / frame-ancestors

Strict-Transport-Security

Referrer-Policy

Permissions-Policy

Cross-Origin policies

Use strict CORS rather than wildcard CORS for authenticated browser applications.

48. Frontend Architecture

Use a modern framework such as:

Next.js / React

or equivalent.

Recommended structure:

apps/
 web/
 marketing
 customer
 admin/
 superadmin

The customer and admin applications may share component libraries/design systems, but their authorization boundaries must remain separate.

49. Customer Dashboard

Must provide:

Overview
WhatsApp sessions
QR/pairing
Calls
Messages
API credentials
Webhooks
Billing
Usage
Account settings
Security
Active browser sessions

50. Marketing Site

Public site should include:

Product
Features
Pricing
Documentation
Security
Terms
Privacy
Sign in
Sign up

Do not mix authenticated application state into the public marketing surface.

51. OpenAPI

Keep OpenAPI.

But CI must verify:

OpenAPI

↔

actual API

Add:

- schema validation
 - request validation
 - response validation where practical
 - compatibility testing
 - API versioning rules
-

52. API Error Contract

Use a consistent format.

Example:

```
{
  "error": {
    "code": "SESSION_NOT_FOUND",
    "message": "The requested session does not exist.",
    "request_id": "req_123"
  }
}
```

Never expose:

- SQL errors
 - stack traces
 - internal paths
 - credentials
 - protocol internals
-

53. Pagination

Use cursor-based pagination for large collections such as:

- calls
- messages
- audit logs
- webhook deliveries
- sessions

Define consistent:

- cursor
- limit
- next_cursor

behavior.

54. Data Retention

Define explicit retention policies for:

- messages
- call records
- audit events
- webhook payloads
- webhook attempts
- logs
- metrics
- uploaded media
- auth sessions
- WhatsApp device state

Do not retain raw call audio by default unless explicitly required as a product feature.

Sensitive communication data should not accidentally enter logs or backups.

55. Infrastructure

Initial production deployment may use:

Caddy / reverse proxy
API
Realtime worker
PostgreSQL
Redis
Object storage

Dockerize all application services.

Do not introduce Kubernetes merely for the sake of being "enterprise."

The architecture must be Kubernetes-compatible eventually, but Docker Compose is acceptable for the initial deployment.

56. Container Security

Containers should:

- run as non-root
- use minimal images
- drop unnecessary Linux capabilities
- use `no-new-privileges`
- use resource limits
- use health checks
- avoid writable root filesystems where practical
- contain only required dependencies

Separate:

API container
Worker container
Frontend container

where operationally appropriate.

57. Network Architecture

Private infrastructure:

Internet



Reverse Proxy

↓
API
↓
PostgreSQL
Redis
Workers

PostgreSQL and Redis must not be publicly accessible.

Use network-level access control.

58. Secrets Management

Replace plaintext `.env` production secrets with a proper secret-management system.

Possible options:

Cloud provider secret manager
Doppler
Infisical
Vault

Secret lifecycle must include:

creation
rotation
versioning
deployment
revocation
emergency rotation
audit

Maintain an explicit secret inventory.

59. CI/CD

CI must run:

go test ./...
go vet
static analysis
dependency vulnerability scanning

secret scanning
SAST
frontend tests
frontend dependency audit
container scanning

Generate an SBOM.

No secrets may be committed.

Deployment:

Pull Request



CI



Build



Security scan



Staging



Smoke tests



Approval



Production

60. Deployment Strategy

For initial production:

Build image



Deploy staging



Health check



Backup



Deploy production



Migration



Smoke test

Have a documented rollback procedure.

Do not perform uncontrolled in-place binary replacement.

61. Database Backups

PostgreSQL:

- automated backups
- point-in-time recovery where available
- encryption
- retention policy
- restore testing

WhatsApp device store:

- encrypted backup
- separate backup policy
- restore testing

Important:

Losing the WhatsApp device store may require customers to re-link their WhatsApp accounts.

Therefore this store requires explicit disaster-recovery treatment.

62. Disaster Recovery

Define:

RPO

RTO

and test:

database failure

Redis failure

API instance failure

worker failure

VPS failure

database restore

WhatsApp store restore

network failure
Stripe outage
webhook destination outage

63. Failure Behavior

The final system must define behavior for:

PostgreSQL unavailable

No unsafe state mutation.

Redis unavailable

Fail safely; do not bypass authorization or rate limits.

Stripe unavailable

Do not grant paid entitlements.

Worker crash

Lease expires and another worker can recover the session.

WhatsApp disconnect

Transition to reconnecting.

Webhook destination unavailable

Persist and retry.

Media download failure

Cleanly fail without leaking memory/resources.

Full disk

Fail safely and alert.

Memory pressure

Enforce resource limits/backpressure.

64. Implementation Plan

Phase 0 — Emergency Security Containment

Goal: Immediately eliminate currently exploitable vulnerabilities before architectural work.

Tasks:

1. Remove direct customer plan activation.
2. Remove all production-accessible simulated billing activation.
3. Verify Stripe webhook signatures.
4. Make Stripe events idempotent.
5. Require authentication for all session lifecycle endpoints.
6. Fix all session ownership checks.
7. Fix all messaging/calling ownership checks.
8. Fix API-key deletion authorization.
9. Replace SHA-256 password storage.
10. Invalidate/rotate existing weak credentials.
11. Remove hardcoded superadmin.
12. Rotate every credential that may have existed in source control.
13. Rotate Stripe secrets.
14. Rotate master/break-glass credentials.
15. Rotate webhook secrets.
16. Add immediate authentication/call rate limits.
17. Add SSRF protection.
18. Add media size limits.
19. Review production logs for suspicious access.
20. Review billing changes.
21. Review session changes.
22. Review abnormal calls/messages.
23. Deploy from a clean artifact.

Because the existing review identifies a committed superadmin credential as potentially compromised, treat those credentials as already compromised rather than merely "unsafe."

Phase 0 acceptance gate

All critical vulnerabilities identified in the security review must have:

fix
+
regression test
+
verification

before proceeding.

Phase 1 — Domain Model, Database & Architecture Foundation

Goal: Establish the correct structural foundation.

Implement:

- PostgreSQL
- migration framework
- organization/tenant model
- memberships
- identity model
- authentication-session model
- API credential model
- WhatsApp session metadata
- calls
- messages
- subscriptions
- entitlements
- usage
- audit events
- webhook events
- outbox events
- feature flags

Separate:

Application DB

from:

WhatsApp device store

Add:

- foreign keys
- indexes
- constraints
- transaction boundaries
- connection pooling
- statement timeouts
- backup configuration

Create architecture decision records for:

control plane
real-time plane
multi-tenancy
authentication
authorization
worker ownership
event architecture
billing
observability
disaster recovery

Acceptance gate

- migrations run cleanly
 - schema constraints verified
 - indexes verified
 - tenant model implemented
 - application DB separated from WhatsApp store
 - existing data migration plan tested against a copy
 - no existing WhatsApp credentials are lost
-

Phase 2 — Authentication & Identity

Goal: Build a secure identity system.

Implement:

- Argon2id/bcrypt
- email verification
- password reset
- login protection
- browser sessions
- session revocation
- logout everywhere
- Google OIDC
- PKCE
- MFA
- CSRF protection
- secure cookies
- API credential issuance
- API credential hashing
- API scopes
- credential revocation
- credential expiration

Superadmin:

- separate bootstrap
- mandatory MFA
- short sessions
- step-up authentication

Acceptance gate

Test:

login

logout

logout-all

password reset

email verification

Google login

Google account linking

MFA

credential revocation

session expiry

CSRF

OAuth state/PKCE

No browser authentication credential may exist in `localStorage`.

Phase 3 — Authorization & Multi-Tenancy

Goal: Make tenant isolation mathematically and operationally explicit.

Implement:

Organization

Membership

Role

Permission

Resource authorization

Create a centralized authorization layer.

Every request must resolve:

Principal

↓

Organization

↓

Resource



Action

No handler may directly assume that possessing a resource ID grants access.

Test every resource:

User A cannot access User B

Organization A cannot access Organization B

Developer cannot access admin

Viewer cannot modify

Revoked credential cannot operate

Suspended organization cannot perform operations

Acceptance gate

Cross-tenant authorization tests must cover every sensitive endpoint.

No endpoint is considered complete without an authorization test.

Phase 4 — Real-Time Worker & WhatsApp Architecture

Goal: Separate the control plane from the stateful WhatsApp/VoIP runtime.

Create:

Session Registry

Worker Manager

Worker Lease

Worker Heartbeat

Command Routing

Event Routing

Move the architecture toward:

API



command



worker



WhatsApp

Workers own:

whatsmeow.Client
Session
CallManager
CallContext
signaling
relay
RTP/SRTP
MLow
media

Implement:

- worker lifecycle
- lease ownership
- fencing
- reconnect
- worker crash recovery
- graceful shutdown
- session reassignment

Do not transport raw audio through Redis.

Redis should handle coordination/control-plane data.

VoIP security gate

Validate:

- SRTP authentication
- replay protection
- RTP robustness
- malformed packets
- sequence rollover
- ROC handling
- race safety
- memory safety
- codec regression

Acceptance gate

Kill a worker during:

idle session
session connection
incoming call
outgoing call
active media

and verify safe recovery behavior.

Phase 5 — Eventing, Outbox, Webhooks & API Reliability

Goal: Make asynchronous behavior durable.

Implement:

Domain Events
Commands
Transactional Outbox
Webhook Delivery
Retries
Dead Letter
Idempotency

State transitions and event creation must be atomic.

Implement webhook:

PENDING
DELIVERING
RETRY_WAIT
DELIVERED
FAILED
DEAD_LETTER

Implement:

- HMAC signatures
- timestamp validation
- replay protection
- exponential backoff
- jitter
- endpoint health
- automatic disabling after repeated failure
- delivery history

Implement API idempotency for important mutation endpoints.

Acceptance gate

Simulate:

process crash during DB transaction
process crash after DB transaction
webhook destination outage
duplicate webhook
duplicate API request
worker restart

No critical event may silently disappear.

Phase 6 — API Security, Media Security & Abuse Prevention

Goal: Harden all external attack surfaces.

Implement:

- rate limiting
- concurrency limits
- usage quotas
- request-size limits
- response-size limits
- pagination
- API scopes
- secure outbound HTTP
- SSRF protection
- media validation
- object storage
- WebSocket hardening
- WebRTC hardening
- API error contracts
- security headers
- CORS policy
- request IDs

Implement communications abuse controls:

calls/minute
concurrent calls
messages/minute
session limits
WebSocket limits
media limits

Acceptance gate

Security tests must cover:

BOLA
IDOR
SSRF
XSS
CSRF
credential theft
rate-limit bypass
resource exhaustion
WebSocket authorization
WebRTC authorization
malformed media

Phase 7 — Billing, Entitlements & Usage

Goal: Make billing authoritative and tamper-resistant.

Implement:

Stripe

↓

Verified Events

↓

Subscription

↓

Entitlement

↓

Usage

Implement:

- Stripe signature verification
- event deduplication
- reconciliation
- cancellation
- failed payment handling
- proration
- dunning
- usage tracking
- entitlement enforcement
- manual admin overrides
- override expiration

Never allow:

customer → plan

directly.

Acceptance gate

Test:

- successful payment
- failed payment
- duplicate Stripe event
- forged Stripe event
- subscription cancellation
- subscription downgrade
- Stripe outage
- manual admin grant
- expired manual grant

Phase 8 — Frontend & Superadmin Operations Platform

Goal: Replace the embedded frontend with production-grade interfaces.

Build:

- Public Marketing
- Customer Dashboard
- Superadmin Dashboard

Customer dashboard:

- sessions
- QR pairing
- calls
- messages
- API credentials
- webhooks
- billing
- usage
- security
- browser sessions

Superadmin:

- users
- organizations
- subscriptions
- entitlements

sessions
workers
calls
webhooks
audit
platform health
feature flags
support access

Implement:

- safe rendering
- CSP
- secure cookies
- CSRF
- CORS
- responsive UI
- loading/error states
- pagination
- filtering
- search

Acceptance gate

No direct unsafe `innerHTML` rendering of untrusted server data.

No customer API credential is exposed through the admin interface.

Phase 9 — Observability, Infrastructure, Disaster Recovery & Production Certification

Goal: Make the system operable as a real production service.

Implement:

Infrastructure

- Docker
- reverse proxy
- TLS
- PostgreSQL
- Redis
- object storage
- worker containers
- API containers
- private networking

CI/CD

- unit tests
- integration tests
- authorization tests
- security scans
- dependency scans
- secret scans
- container scans
- SBOM
- staging deployment
- production approval
- rollback

Observability

- structured logs
- metrics
- tracing
- Sentry/error tracking
- dashboards
- alerts
- request correlation

Operations

Define:

SLO

SLI

RPO

RTO

Create:

- incident response runbook
- deployment runbook
- rollback runbook
- database restore runbook
- WhatsApp-store restore runbook
- credential rotation runbook
- security incident runbook

74. Final Production Certification

The system must not be declared production-ready until all of the following pass.

Security

- ☐ No known critical vulnerabilities
- ☐ High-risk vulnerabilities reviewed
- ☐ Penetration test completed
- ☐ Secrets removed from repository
- ☐ All exposed secrets rotated
- ☐ SSRF protection tested
- ☐ XSS protection tested
- ☐ CSRF protection tested
- ☐ BOLA tests pass
- ☐ API credential tests pass
- ☐ WebSocket authorization tests pass
- ☐ WebRTC authorization tests pass

Authentication

- ☐ Email verification
- ☐ Password reset
- ☐ Strong password hashing
- ☐ Browser session expiry
- ☐ Logout
- ☐ Logout all
- ☐ OAuth/OIDC
- ☐ PKCE
- ☐ MFA
- ☐ Superadmin MFA
- ☐ Credential revocation

Authorization

- ☐ Organization isolation
- ☐ Cross-tenant tests
- ☐ RBAC tests
- ☐ Admin authorization
- ☐ Resource-level authorization
- ☐ Default deny

WhatsApp

- ☐ Session ownership
- ☐ Worker lease
- ☐ Worker failover
- ☐ Reconnect
- ☐ Credential backup

- ☐ Credential restore
- ☐ No credential leakage

VoIP

- ☐ RTP tests
- ☐ SRTP authentication
- ☐ Replay protection
- ☐ ROC tests
- ☐ malformed packet tests
- ☐ race detector
- ☐ codec regression tests
- ☐ call lifecycle tests

Billing

- ☐ Stripe signature verification
- ☐ Stripe event deduplication
- ☐ reconciliation
- ☐ no client-controlled entitlement
- ☐ manual override audit
- ☐ usage enforcement

Webhooks

- ☐ durable outbox
- ☐ retries
- ☐ exponential backoff
- ☐ dead-letter handling
- ☐ signatures
- ☐ replay protection
- ☐ idempotency

Reliability

- ☐ API failure tested
- ☐ database failure tested
- ☐ Redis failure tested
- ☐ worker failure tested
- ☐ WhatsApp disconnect tested
- ☐ webhook outage tested
- ☐ Stripe outage tested
- ☐ graceful shutdown tested

Disaster Recovery

- ☐ RPO defined

- ☐ RTO defined
- ☐ PostgreSQL backup tested
- ☐ PostgreSQL restore tested
- ☐ WhatsApp store backup tested
- ☐ WhatsApp store restore tested
- ☐ production rollback tested

Performance

- ☐ API load test
 - ☐ concurrent session test
 - ☐ concurrent call test
 - ☐ WebSocket load test
 - ☐ webhook load test
 - ☐ database load test
 - ☐ memory profiling
 - ☐ goroutine profiling
-

75. Engineering Rules for Antigravity

These rules apply to the entire implementation.

Rule 1 — Do not rewrite the VoIP engine unnecessarily

Preserve:

internal/voip/**
internal/wa/**
MLow
RTP
SRTP
STUN
signaling
relay

unless a verified security/correctness problem requires modification.

Rule 2 — Do not perform architecture-wide changes in one step

Every phase must:

inspect
→ plan
→ implement
→ test
→ review
→ document

before moving forward.

Rule 3 — Never silently weaken security to make a feature work

Examples:

Do not:

disable authorization
disable CSRF
allow anonymous access
skip Stripe verification
bypass rate limits
trust arbitrary URLs
store secrets temporarily in plaintext

as "temporary" implementation shortcuts.

Rule 4 — No direct SQL from HTTP handlers

Use:

Handler
↓
Application Service
↓
Repository / Port
↓
Infrastructure

Rule 5 — No direct WhatsApp access from HTTP handlers

Use:

Handler



Application Service



Command



Worker



WhatsApp

Rule 6 — Authorization is mandatory for every resource

Never trust:

resource ID

session ID

call ID

API key ID

user ID

organization ID

provided by a client.

Always resolve ownership through the authenticated principal.

Rule 7 — Deny by default

If authorization is uncertain:

DENY

Rule 8 — No secrets in logs

Never log:

passwords

PATs

API keys

cookies

OAuth tokens

Stripe secrets

webhook secrets
WhatsApp credentials

Rule 9 — Every security fix requires a regression test

If a bug was:

cross-tenant access

the final code must contain a test proving:

cross-tenant access fails

Rule 10 — Every phase must leave the repository buildable

Do not leave the repository in a permanently broken intermediate state.

76. Required Development Workflow

For each phase:

Step 1 — Repository inspection

Before changing code, inspect the relevant current implementation.

Step 2 — Architecture plan

Document:

files affected
database changes
API changes
security impact
migration impact
rollback strategy

Step 3 — Implementation

Implement only the current phase.

Step 4 — Tests

Run:

```
go test ./...
```

plus relevant integration/security tests.

Where applicable:

```
go test -race ./...
```

Step 5 — Static/security checks

Run available:

```
go vet
staticcheck
gosec
govulncheck
dependency scanning
secret scanning
```

Step 6 — Review

Review:

```
git diff
database migration
API behavior
security boundaries
resource ownership
```

Step 7 — Phase report

Report:

```
implemented
changed files
database changes
API changes
tests
known issues
migration instructions
rollback procedure
```

Step 8 — Stop

Do not automatically begin the next high-risk phase until the current phase has passed its acceptance gate.

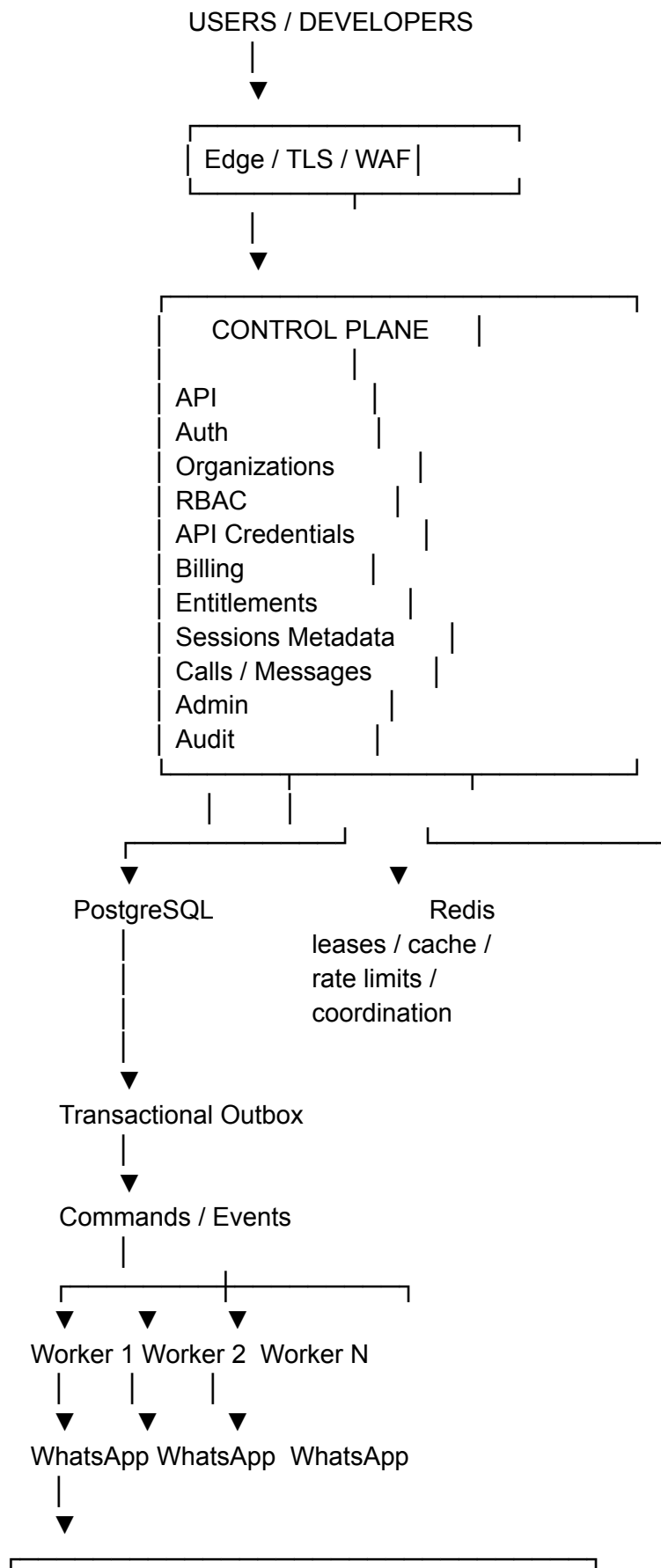
77. Required Documentation Structure

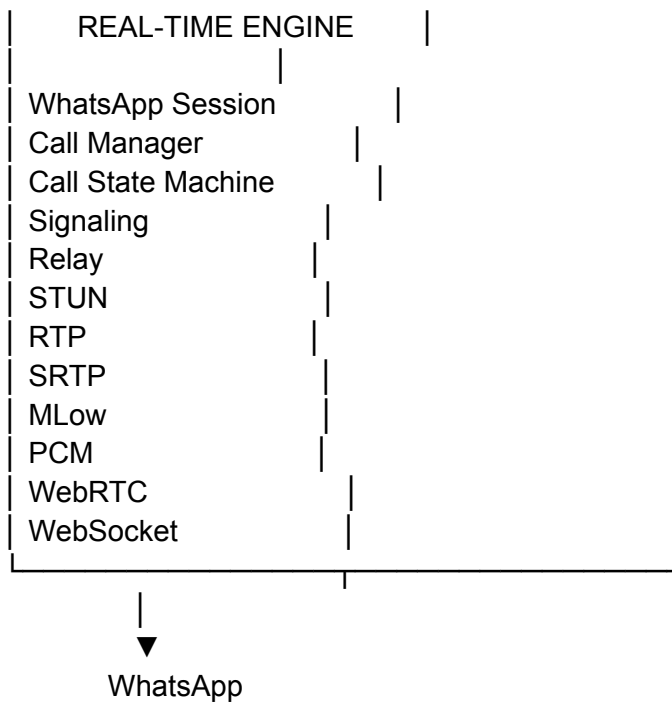
Create:

```
docs/
├── architecture/
│   ├── overview.md
│   ├── control-plane.md
│   ├── realtime-plane.md
│   ├── multitenancy.md
│   ├── authentication.md
│   ├── authorization.md
│   ├── worker-architecture.md
│   ├── event-architecture.md
│   ├── billing.md
│   ├── observability.md
│   └── disaster-recovery.md
├── adr/
│   ├── ADR-001-...
│   ├── ADR-002-...
│   └── ...
├── operations/
│   ├── deployment.md
│   ├── rollback.md
│   ├── incident-response.md
│   ├── secret-rotation.md
│   ├── database-restore.md
│   └── whatsapp-store-restore.md
└── security/
    ├── threat-model.md
    ├── authentication.md
    ├── authorization.md
    └── security-testing.md
```

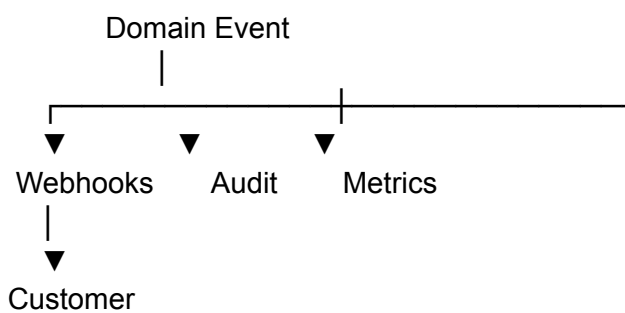
78. Final Target Architecture

The completed system should conceptually look like this:





And asynchronously:



79. Final Objective

The final WacallerAPI system should no longer be architecturally equivalent to:

Single Go process

+

in-memory sessions

+

SQLite

+

large HTTP handler layer

+

best-effort webhooks

+

permanent PAT

+

user-based authorization

It should become:

Secure Multi-Tenant Control Plane

+

Durable Event Architecture

+

Stateful WhatsApp Worker Plane

+

Strong Resource Authorization

+

Secure Identity

+

Stripe-backed Entitlements

+

Reliable Webhooks

+

Production VoIP Security

+

Observable Infrastructure

+

Tested Disaster Recovery

The most important architectural principle is:

**Separate the control plane from the real-time communications plane
without unnecessarily rewriting the existing VoIP engine.**

The control plane manages **who is allowed to do what**.

The real-time plane manages **how WhatsApp calls and media actually happen**.

The database stores durable business state.

The event/outbox system connects durable state transitions to asynchronous processing.

Workers own live WhatsApp connections.

Redis coordinates distributed runtime state but does **not** carry raw audio.

PostgreSQL is the source of truth for application state.

Stripe is the source of truth for billing events.

The authorization layer is the source of truth for resource access.

And every security-sensitive operation must be auditable.

80. Completion Criteria

Do not consider the project "enterprise production ready" merely because:

PostgreSQL works
Redis works
Docker works
Next.js works
Stripe works
MFA works

The system is complete only when the **entire architecture works under failure, attack, concurrency, migration, and recovery conditions.**

The final certification must demonstrate:

secure
+
tenant-isolated
+
authorized
+
durable
+
observable
+
recoverable
+
rate-limited
+
idempotent
+
auditable
+
load-tested
+
security-tested

while preserving the existing WacallerAPI WhatsApp/VoIP capabilities.

Implement strictly in the 10 phases above. Do not skip acceptance gates. Do not make unrelated architectural changes during a phase. Do not rewrite functioning VoIP/MLow/protocol components without a demonstrated technical reason.